CPSC 100

Computational Thinking

Generative Artificial Intelligence

Instructor: Parsa Rajabi
Department of Computer Science
University of British Columbia

Agenda

- About Me
- Generative AI
  - Background
  - Large Language Model (LLM)
- Co-creating a CPSC 100 AI policy
  - Class Activity

Learning Goals

Learning Goals

After this lecture, you should be able to:

- Explain the concept of **Generative** **A**rtificial **I**ntelligence
  - Describe the relevance of Generative AI in CPSC 100
- Identify and list various GenAI tools to date
- Identify opportunities to use and not use GenAI
- Develop a classroom AI policy

Generative AI

Generative AI

What does Generative mean?

Generative refers to systems, models, or algorithms designed to **create or produce new data** that **resembles the input data** on which they were trained.

Generative AI

What does AI mean?

Artificial Intelligence (AI), which refers to the **simulation of human intelligence** in machines that are programmed to think, reason, learn, and **act in a way** that mimics human cognitive abilities

Background Information

Hello ChatGPT!

- Previous conversational chatbots (~1960-2000s)
- ELIZA[1], PARRY[2], A.L.I.C.E.[3] & Cleverbot [4]
- Rise of ChatGPT-3.5 in November 2022
  - Chat **G**enerative **P**re-trained **T**ransformer
- Wide range of capabilities
  - Human-like responses [5]
  - Understanding conversation context [6]
  - Writing code & debugging [7]

Chatbots are NOT new!

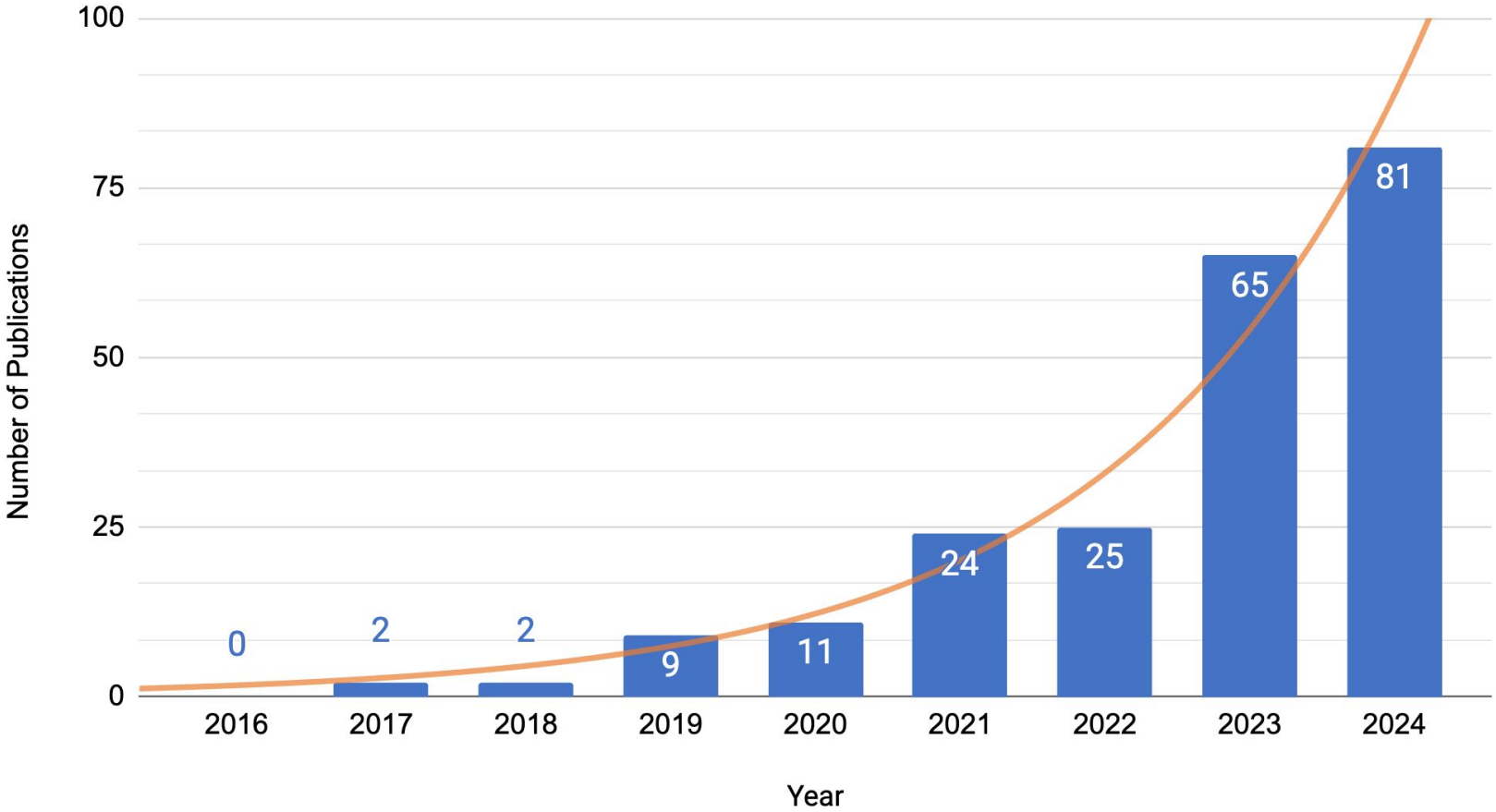
Unleashing the Potential of Chatbots in Education: A State-Of-The-Art Analysis

Rainer Winkler and Matthias Soellner March 2018

psychology literature before examining and individually coding a relevant subset of 80 articles.

The results show that chatbots are in the very beginning of entering education. Few studies suggest

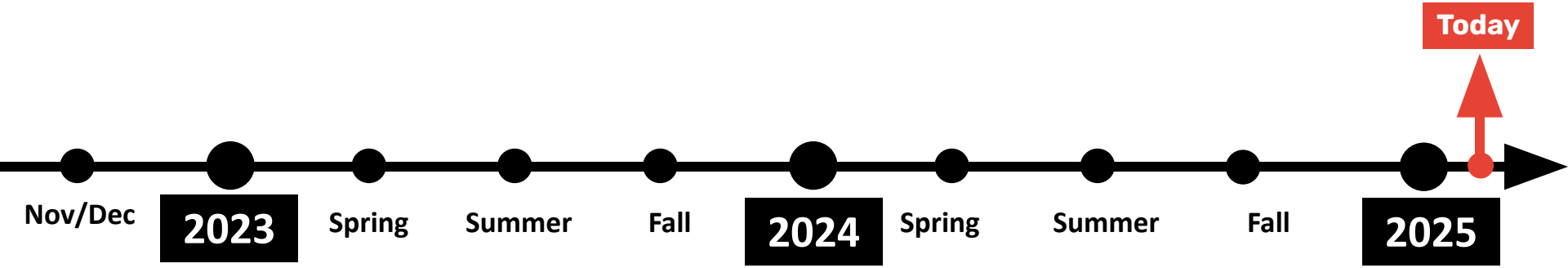
the potential of chatbots for improving learning processes and outcomes. Nevertheless, past


Occurrence of keyword "Chatbot in Education" in 2016-2024
Number of Publications
100
75
50
25
0
0
2
2
9
11
24
25
65
81
2016
2017
2018
2019
2020
2021
2022
2023
2024
Year

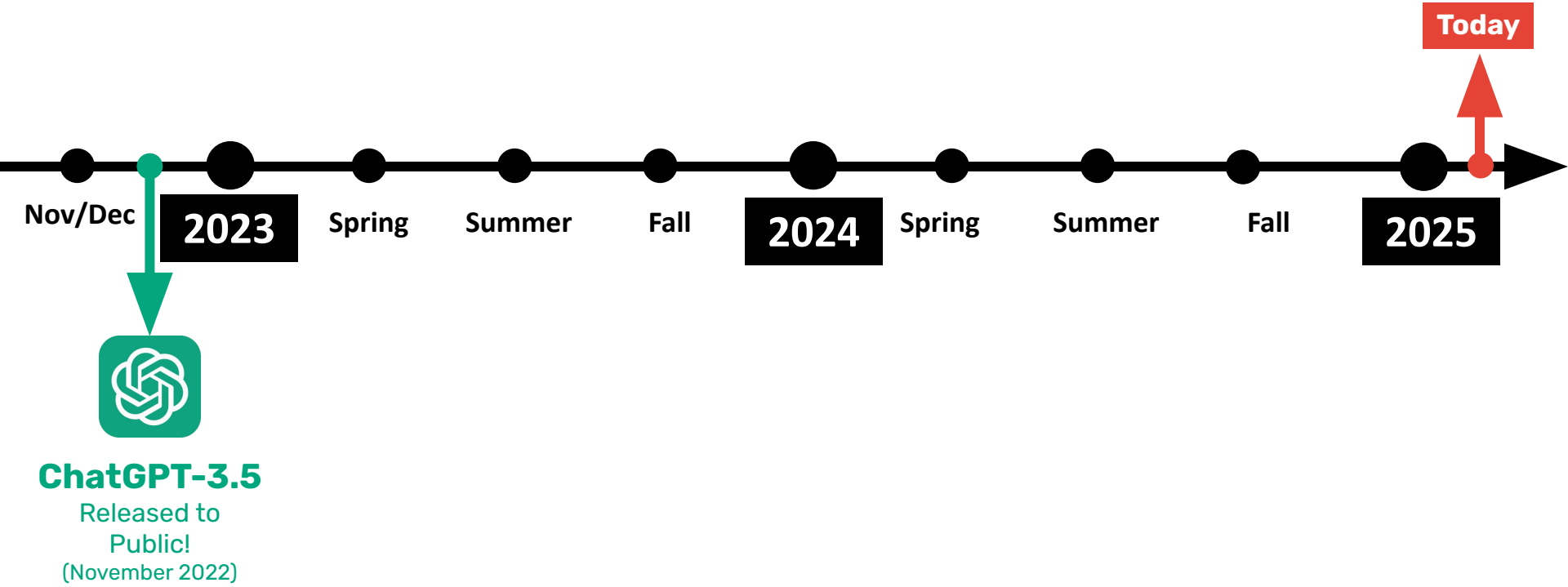
Timeline

Timeline

Timeline



Timeline


Today
Nov/Dec
2023
Spring
Summer
Fall
2024
Spring
Summer
Fall
2025
ChatGPT-3.5
Released to Public!
(November 2022)

Timeline

GPT-4
(March 2023)

Today

Nov/Dec | 2023 | Spring | Summer | Fall | 2024 | Spring | Summer | Fall | 2025

ChatGPT-3.5
Released to Public!
(November 2022)

Timeline

GPT-4
(March 2023)

Gemini
(Dec 2023)

Today

Nov/Dec | 2023 | Spring | Summer | Fall | 2024 | Spring | Summer | Fall | 2025

ChatGPT-3.5
Released to Public!
(November 2022)

Timeline

GPT-4
(March 2023)

Gemini
(Dec 2023)

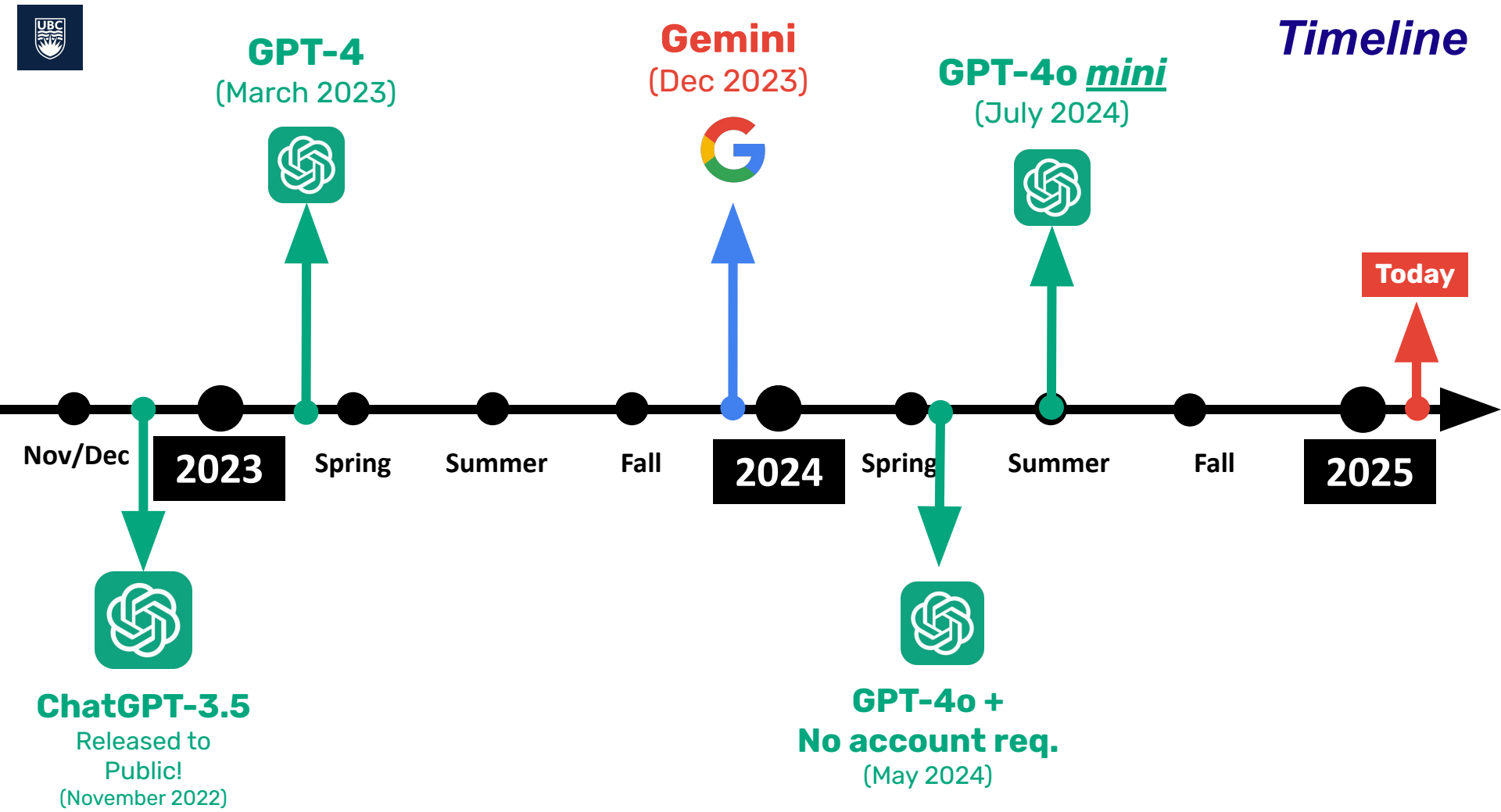
Today

Nov/Dec | 2023 | Spring | Summer | Fall | 2024 | Spring | Summer | Fall | 2025

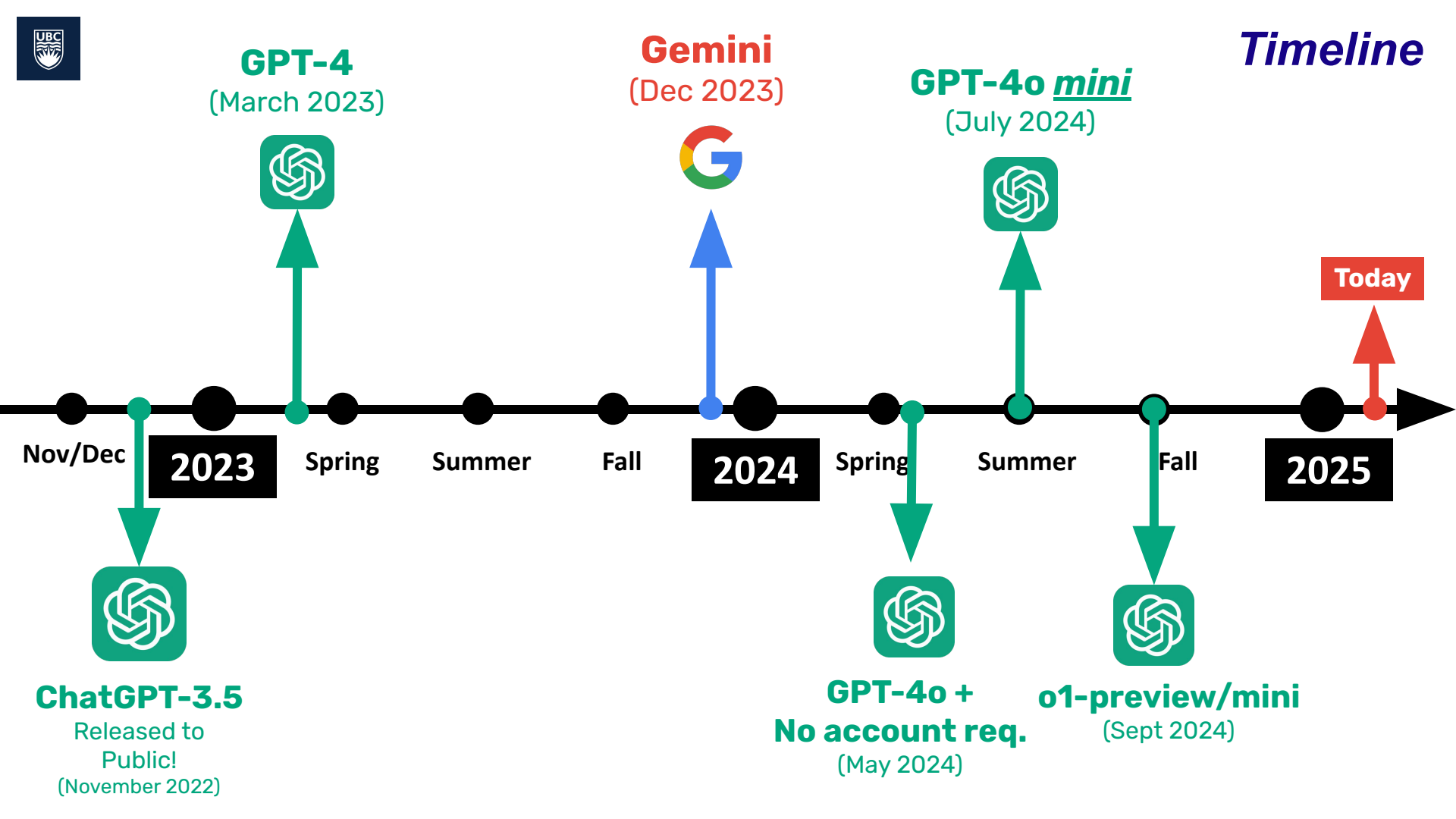
ChatGPT-3.5
Released to Public!
(November 2022)

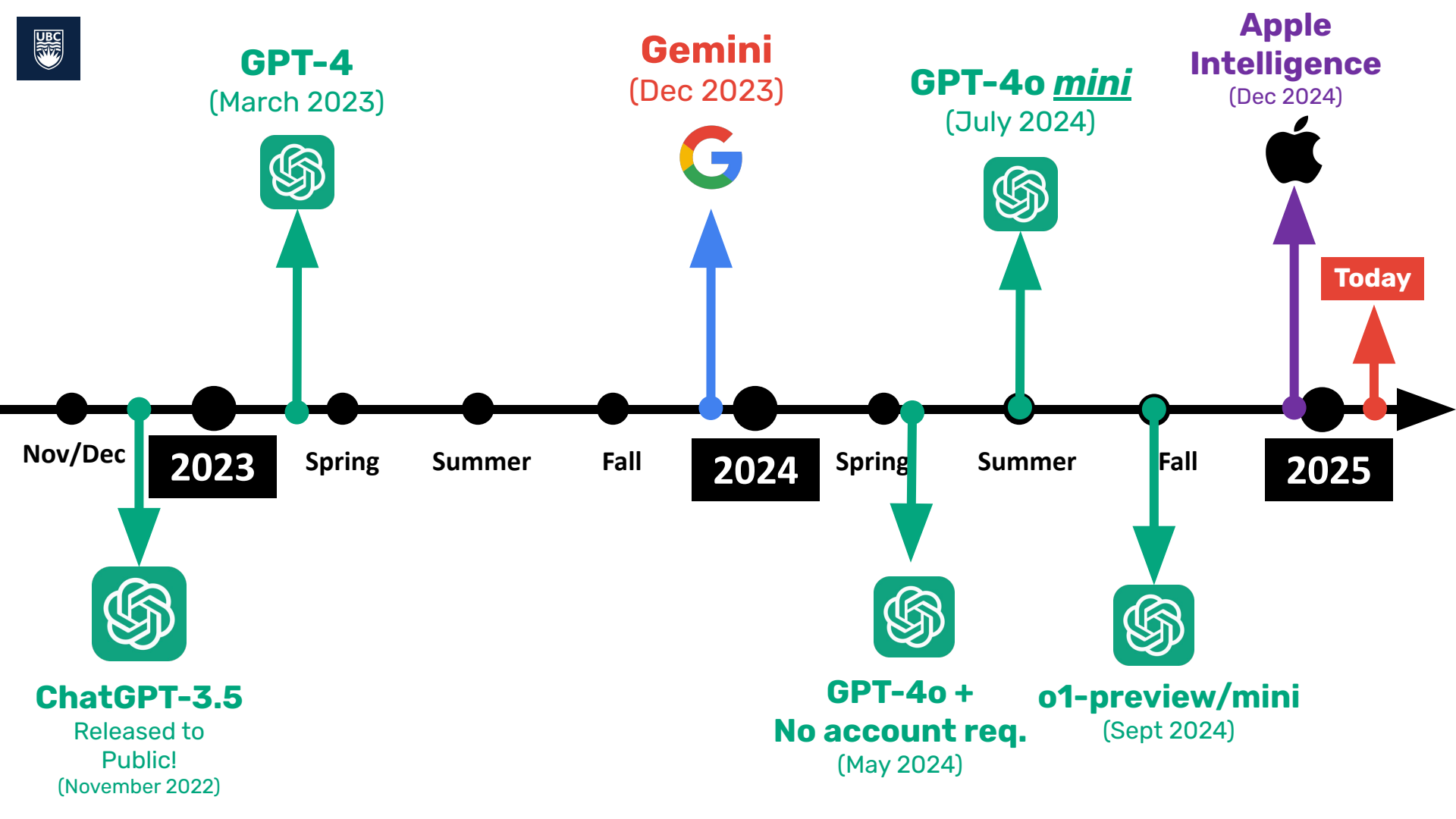
GPT-4o +
No account req.
(May 2024)

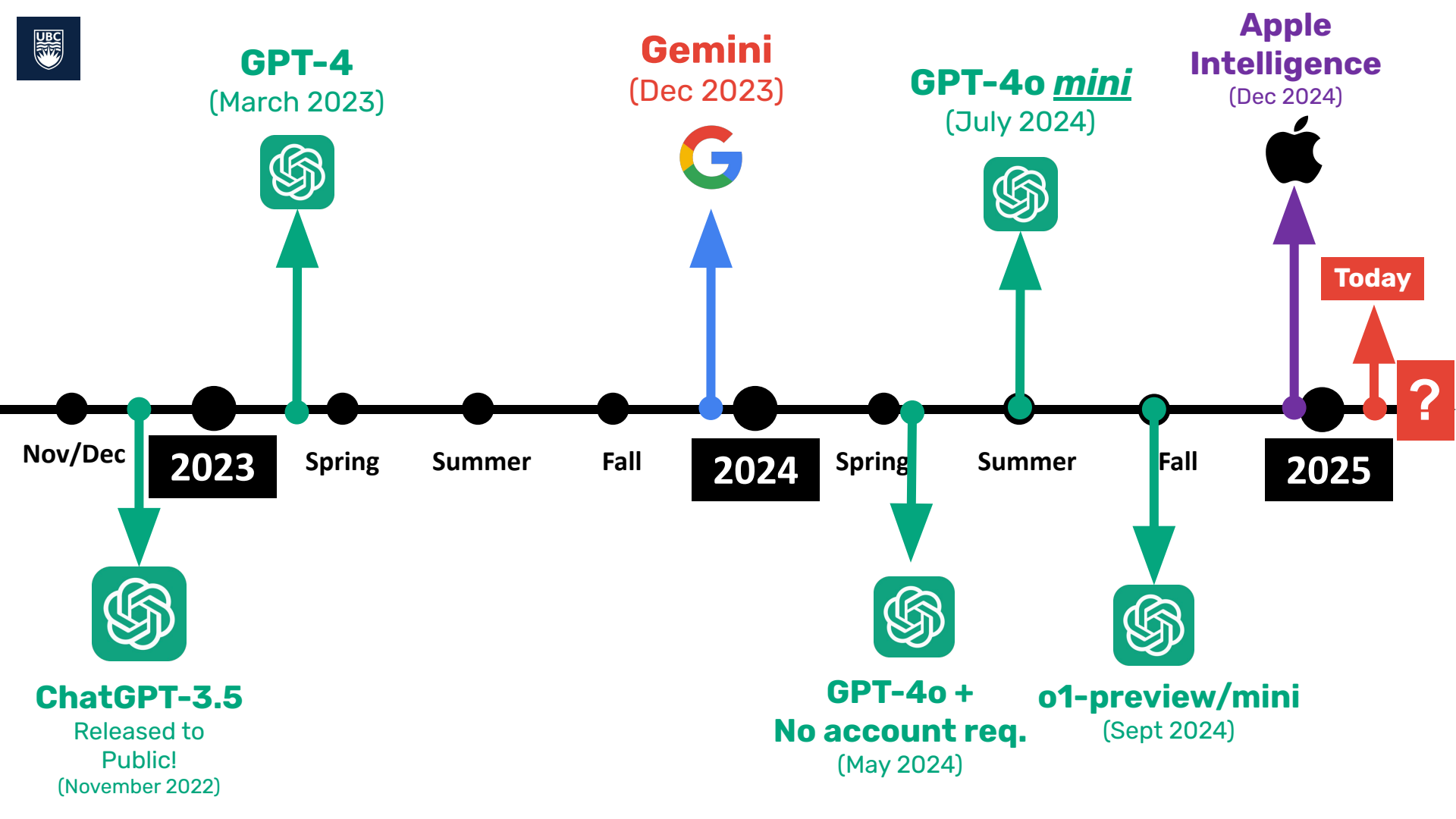
Timeline


GPT-4
(March 2023)
Gemini
(Dec 2023)
GPT-4o mini
(July 2024)
Today
Nov/Dec
2023
Spring
Summer
Fall
2024
Spring
Summer
Fall
2025
ChatGPT-3.5
Released to Public!
(November 2022)
GPT-4o +
No account req.
(May 2024)

Timeline


GPT-4
(March 2023)
Gemini
(Dec 2023)
GPT-4o mini
(July 2024)
Today
Nov/Dec
2023
Spring
Summer
Fall
2024
Spring
Summer
Fall
2025
ChatGPT-3.5
Released to Public!
(November 2022)
GPT-4o +
No account req.
(May 2024)
o1-preview/mini
(Sept 2024)


GPT-4
(March 2023)
Gemini
(Dec 2023)
GPT-4o mini
(July 2024)
Apple Intelligence
(Dec 2024)
Today
Nov/Dec
2023
Spring
Summer
Fall
2024
Spring
Summer
Fall
2025
ChatGPT-3.5
Released to Public!
(November 2022)
GPT-4o +
No account req.
(May 2024)
o1-preview/mini
(Sept 2024)


GPT-4
(March 2023)
Gemini
(Dec 2023)
GPT-4o mini
(July 2024)
Apple Intelligence
(Dec 2024)
Today
?
Nov/Dec
2023
Spring
Summer
Fall
2024
Spring
Summer
Fall
2025
ChatGPT-3.5
Released to Public!
(November 2022)
GPT-4o +
No account req.
(May 2024)
o1-preview/mini
(Sept 2024)

Linking Generative AI to Algorithms

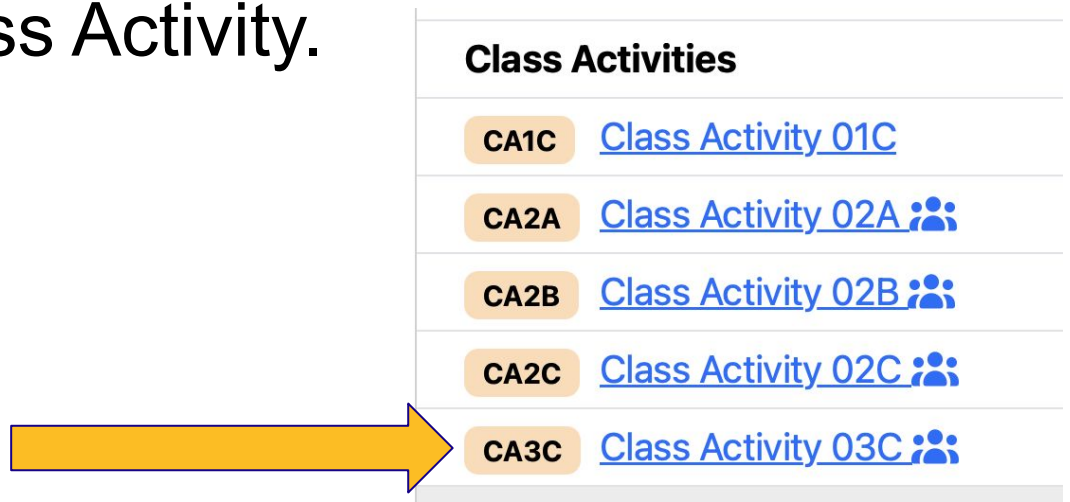
CPSC 100 AI Policy

Class Activity

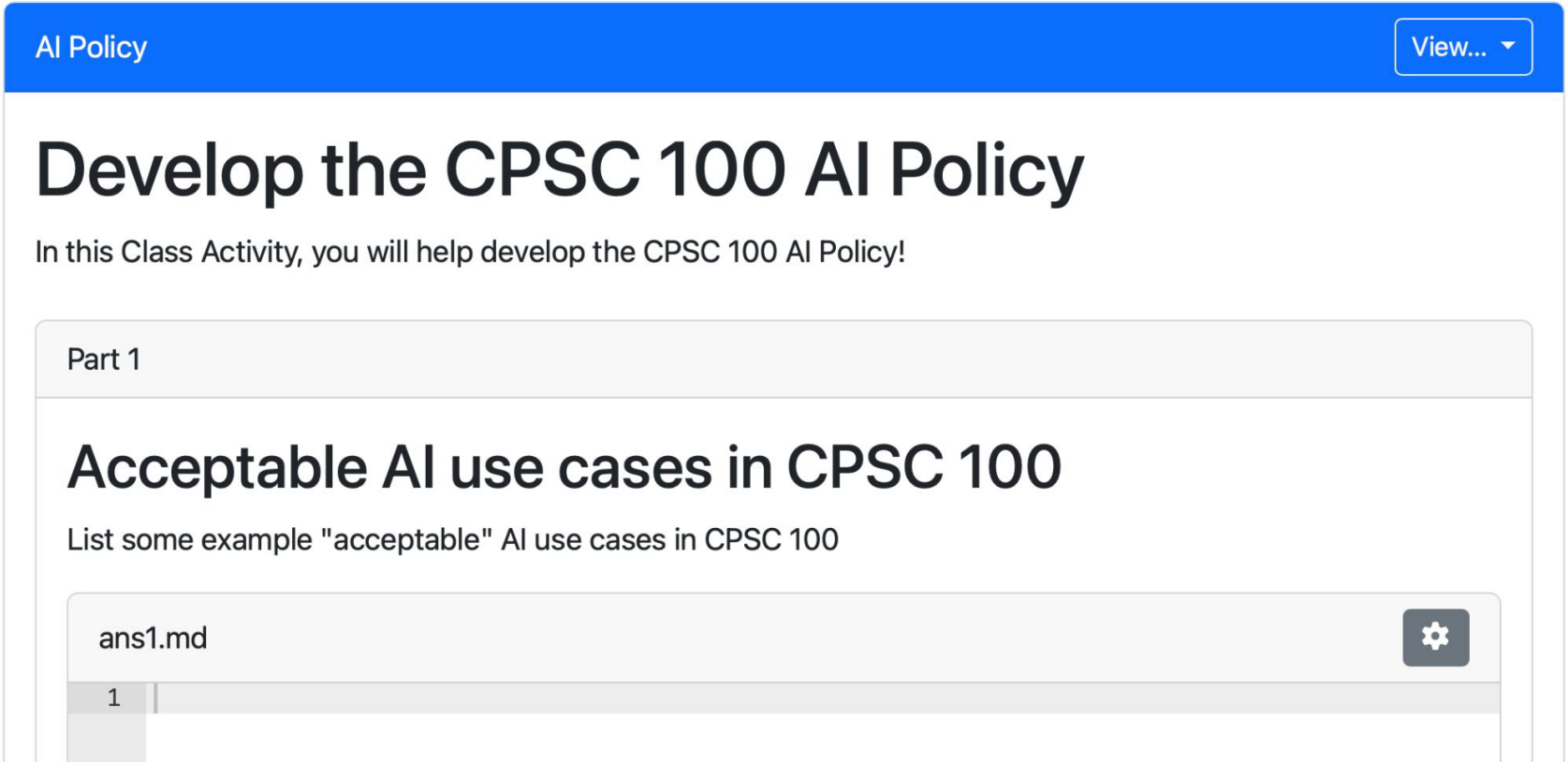
Class Activity: Course AI Policy

As a class, let's identify how AI can or can not be used in a responsible and effective way.

In groups of 2-4, brainstorm ideas on the PrairieLearn Class Activity.



Class Activity: Course AI Policy



Class Activity: Course AI Policy

Part 2

Unacceptable AI use cases in CPSC 100

List some example "unacceptable" AI use cases in CPSC 100

ans2.md

```
1
```

Class Activity: Course AI Policy

Part 3

Risks of using AI in CPSC 100

Write down any risks of using AI in CPSC 100.

ans3.md

```
1
```

Class Activity: Course AI Policy

Part 4

Opportunities of using AI in CPSC 100

Write down any opportunities of using AI in CPSC 100.

ans4.md

```
1
```

Class Discussion

Wrap up

Preview: Programming

This is *not* a programming courses

But you do need to *understand* how programs work

We'll cover a small amount of basic concepts in class and you'll work on a visual language in lab

From algorithms to code: How do programs work?

How do programs work?

Programs are a way of encoding ***algorithms*** in a precise enough way for computers to understand the instructions.

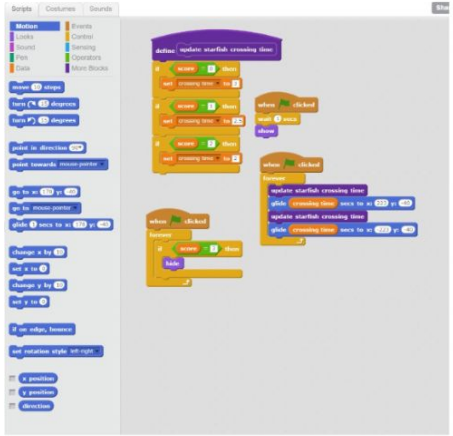
How do programs work?

Programs are a way of encoding ***algorithms*** in a precise enough way for computers to understand the instructions.

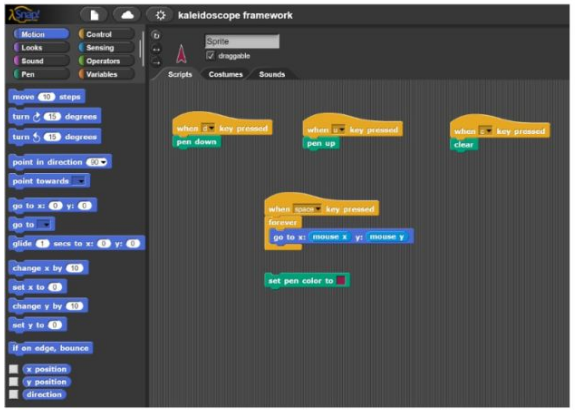
Programmers use a **high level language** like Snap, Scratch, Python, C++, Java, Racket, etc.

These languages may look very different

```
File  Edit  View  Language  Racket  Insert  Tabs  Help
Untitled (define ...) Save        Check Syntax  Debug  Macro Stepper  Run  Stop
(define first car)
(define rest cdr)

(define (addWithCarry x y carry)
  (cond
    ((and (null? x)(null? y)) (if (= carry 0) '() '(1)))
    ((null? x) (addWithCarry '(0) y carry))
    ((null? y) (addWithCarry x '(0) carry))
    ( #t  (let ((bit1 (first x))
                (bit2 (first y)))
            (cond
              ((= (+ bit1 bit2 carry) 0) (cons 0 (addWithCarry (rest x) (rest y) 0)))
              ((= (+ bit1 bit2 carry) 1) (cons 1 (addWithCarry (rest x) (rest y) 0)))
              ((= (+ bit1 bit2 carry) 2) (cons 0 (addWithCarry (rest x) (rest y) 1)))
              (   #t                     (cons 1 (addWithCarry (rest x) (rest y) 1))))))))
```



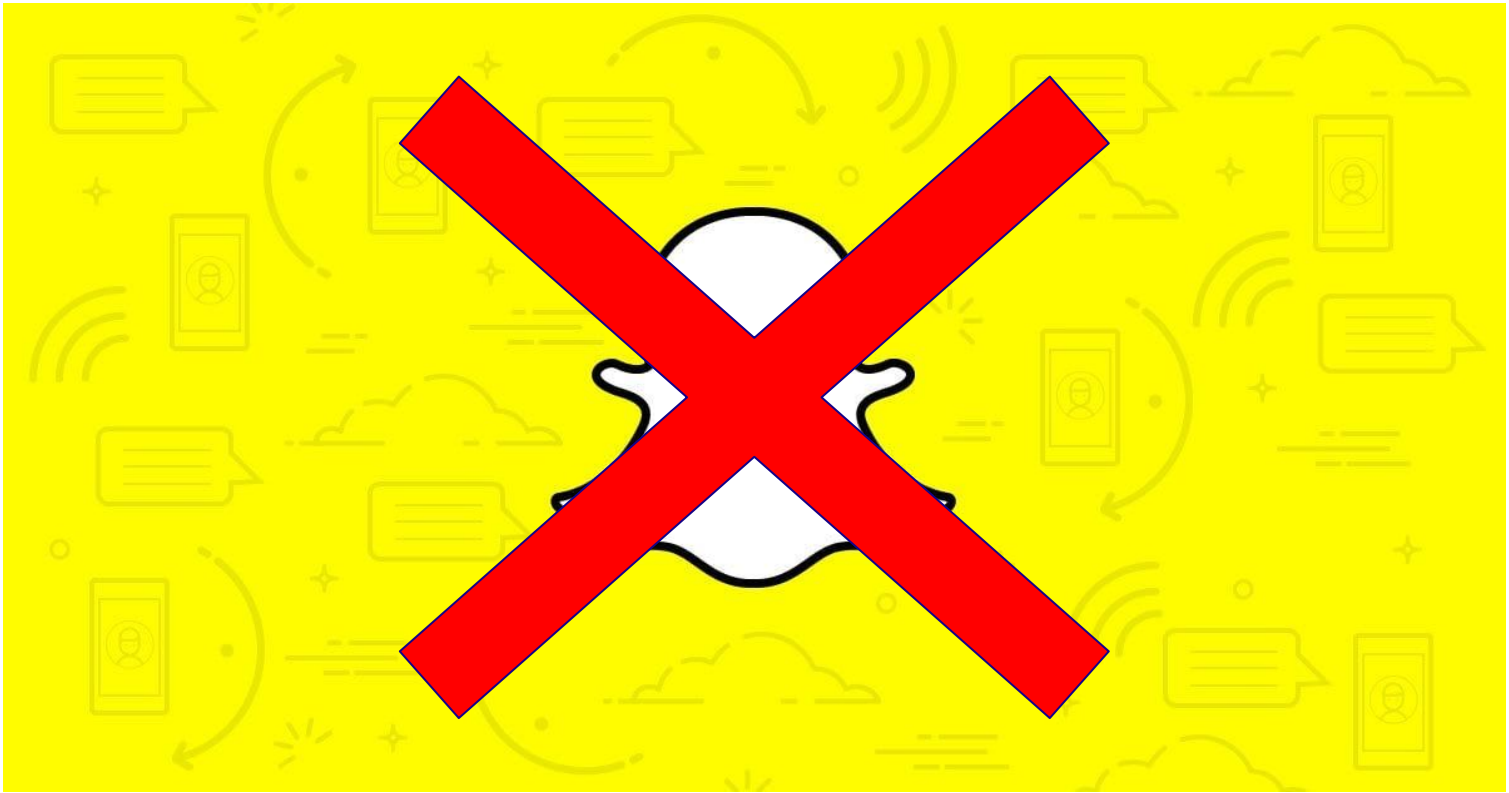
```
/**
 * Simple HelloButton() method.
 * @version 1.0
 * @author john doe <doe.j@example.com>
 */
HelloButton()
{
  JButton hello = new JButton( "Hello, wor
  hello.addActionListener( new HelloBtnList

  // use the JFrame type until support for t
  // new component is finished
  JFrame frame = new JFrame( "Hello Button"
  Container pane = frame.getContentPane();
  pane.add( hello );
  frame.pack();
  frame.show();          // display the fra
}
```




```
    }

    if (bInvokeUI)
    {
        *pbInvokeUI = bInvokeUI;
        *ppwszIdentity = NULL;
        EapTrace("MEapPeerGetIdentity() requesting invoke UI" );
    }
    else
    {
        //GetIdentityToUse( domConnData, domUserData, ppwszIdentity );
    }
```

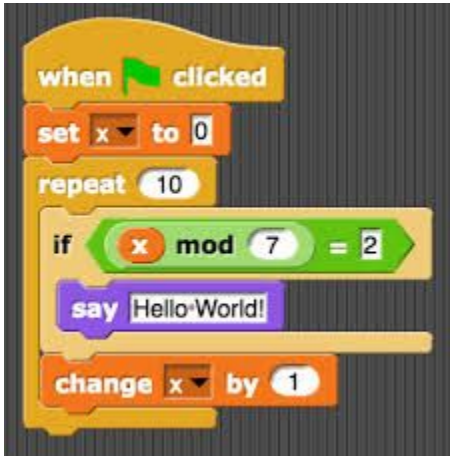
```
def add5(x):
   return x+5

def dotwrite(ast):
    nodename = getNodename()
    label=symbol.sym_name.get(int(ast[0]),ast[0])
    print '    %s [label="%s' % (nodename, label),
    if isinstance(ast[1], str):
       if ast[1].strip():
          print '= %s"];' % ast[1]
       else:
          print '"]'
    else:
       print '"];'
       children = []
       for  in n, childenumerate(ast[1:]):
          children.append(dotwrite(child))
       print ,'    %s -> {' % nodename
       for  in :namechildren
          print '%s' % name,
```

Snap!



Demo


Snap!


when clicked
set x to 0
repeat 10
if x mod 7 = 2
say Hello World!
change x by 1

First, we need to understand how data is represented on computers!